

TV Hub Watch Experience

Flujo MVC completo para consultar un canal desde MongoDB y reproducir su stream con Shaka Player.

Alumno
Stack
Código base
Repositorio público
Entorno de pruebas

Emiliano del Toro

Express 5, TypeScript, Mongoose 8, MongoDB 8.2 (Docker), JavaScript, Shaka Player 4.15

github.com/fcamacho-01fc/tv_hub · rama tv-hub-v4-base

PENDIENTE: pega aquí la URL de tu repositorio público después del push

MacBook Air · Node 22.19 · MongoDB 8.2.12 en el contenedor tvhub-mongo · servidor en localhost:3000

Flujo completado



TODO	Archivo	Solución
1	src/routes/channel.routes.ts	<code>channelRouter.get('/:id', getChannel)</code>
2	src/controllers/channel.controller.ts	<code>Channel.findOne({ _id: channelId, isActive: true })</code>
3	src/controllers/channel.controller.ts	<code>new AppError(404, 'CHANNEL_NOT_FOUND', ...)</code>
4	src/controllers/channel.controller.ts	<code>response.json({ channel })</code>
5	src/public/js/watch.js	<code>fetch(`/api/channels/\${encodeURIComponent(channelId)})`)</code>
6	src/public/js/watch.js	<code>channel.name, channel.country, channel.categories</code>
7	src/public/js/watch.js	<code>new shaka.Player(video) y player.load(channel.streamUrl)</code>
8	src/public/js/watch.js	<code>'playing' tras un load() exitoso, 'error' en el catch</code>

Repositorio público de GitHub

PENDIENTE: pega aquí la URL de tu repositorio público después del push

El repositorio parte del starter oficial (rama `tv-hub-v4-base`) y solo modifica tres archivos de la práctica: `channel.routes.ts`, `channel.controller.ts` y `watch.js`.

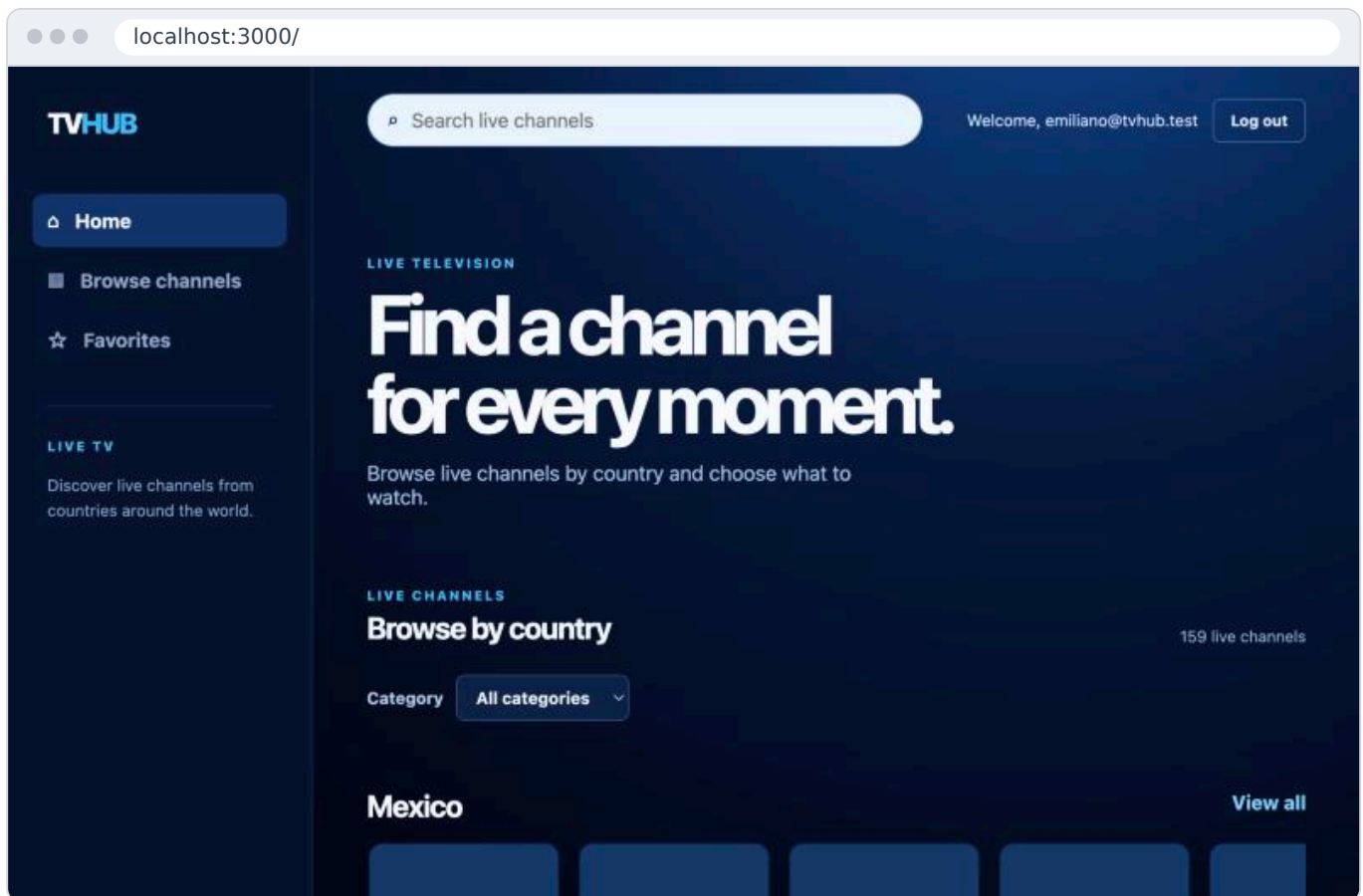
Paso 0 · Preparación con MongoDB

Se levantó MongoDB 8.2 con `docker compose up -d` (contenedor `tvhub-mongo`), se compiló el proyecto y se importaron las playlists de Japón y México. Al iniciar sesión, Home muestra los canales leídos desde MongoDB.

```
Terminal — tv_hub_watch

$ bash entrega/levantar.sh          # extracto de entrega/levantar.log
==> MongoDB con Docker
Docker 29.7.2
docker compose up -d (intento 1)
  Image mongo:8.2 Pulled
  Container tvhub-mongo Started
MongoDB listo
==> Importar canales
$ npm run import:channels -- docs/japon_playlist.m3u Japan
7 channels from Japan processed: 7 created, 0 updated.
$ npm run import:channels -- docs/mexico_playlist.m3u Mexico
159 channels from Mexico processed: 159 created, 0 updated.
```

Evidencia 0.1 · MongoDB levantado en Docker e importación de canales (extracto del log de ejecución).



Evidencia 0.2 · Home con sesión iniciada: 159 canales activos cargados desde MongoDB.

Ejercicio 1 · Route

Qué se hizo. Se conectó `GET /api/channels/:id` con el Controller `getChannel`, que ya estaba importado desde `channel.controller.ts`. La Route no consulta MongoDB. Su única responsabilidad es relacionar el método HTTP y la URL con la función que atiende la solicitud. El segmento `:id` llega al Controller como `request.params.id`.

```
tv_hub_watch / src/routes/channel.routes.ts
1 import { Router } from 'express';
2 import { getChannel, listChannels } from '../controllers/channel.controller.js';
3
4 export const channelRouter = Router();
5
6 channelRouter.get('/', listChannels);
7 // TODO 1:
8 // Completa el Controller que debe atender esta ruta.
9 // Objetivo: conectar GET /api/channels/:id con la lógica que recupera un solo Channel.
10 // Resultado esperado: Express debe ejecutar el Controller correcto cuando se solicita un canal por id.
11 channelRouter.get('/:id', getChannel);
```

Código 1 · TODO 1 resuelto: la ruta dinámica apunta a `getChannel`.

Evidencia de que la Route funciona. Las tres peticiones reciben una respuesta JSON del Controller: 200 con el canal, 404 `CHANNEL_NOT_FOUND` (que solo genera `getChannel`) y 400 por un id mal formado. Además, los tests del proyecto que llaman a `/api/channels/:id` pasan contra MongoDB.

Console · fetch() a la Route desde el navegador (localhost:3000)			
Petición	Status	Content-Type	Significado
GET /api/channels/6aba90e6893465c99e243cfb	200 OK	application/json	Channel encontrado en MongoDB
GET /api/channels/00000000000000000000000000000000	404 Not Found	application/json	CHANNEL_NOT_FOUND (lanzado por getChannel)
GET /api/channels/abc123	400 Bad Request	application/json	INVALID_CHANNEL_ID (validación del id)

Evidencia 1.1 · Respuestas de la Route para un id existente, uno inexistente y uno inválido.

```
tv_hub_watch / tests/channels.test.ts
42 test('GET /api/channels/:id returns one active channel', async () => {
43   const channel = await Channel.findOne({ name: 'Noticias Aula' });
44   const response = await request(app).get('/api/channels/${channel?._id}').expect(200);
45
46   expect(response.body.channel).toEqual(expect.objectContaining({ name: 'Noticias Aula' }));
47 });
48
49 test('GET /api/channels/:id returns the existing error for an unavailable channel', async () => {
50   await request(app)
51     .get('/api/channels/507f1f77bcf86cd799439011')
52     .expect(404, { error: { code: 'CHANNEL_NOT_FOUND', message: 'Channel was not found' } });
53 });
```

```
Terminal — tv_hub_watch · npm test
$ npm test
> tv-hub@0.0.0 test
> jest --runInBand

PASS tests/auth.test.ts (9.569 s)
PASS tests/favorites.test.ts
PASS tests/channels.test.ts
PASS tests/m3u-parser.test.ts
PASS tests/health.test.ts

Test Suites: 5 passed, 5 total
Tests:       16 passed, 16 total
Snapshots:   0 total
Time:        13.586 s
Ran all test suites.
```

Código 1.2 · Tests existentes de la Route: 200 para un canal activo y 404 para uno inexistente.

Evidencia 1.2 · `npm test`: 16/16 tests pasan, incluido `tests/channels.test.ts`.

Ejercicio 2 · Controller

Qué se hizo. En `getChannel` se usó el Channel Model con `findOne()`, porque se necesita un solo documento que cumpla dos condiciones: el `_id` solicitado y `isActive: true`. Si Mongoose no encuentra el documento, regresa `null` y se lanza `AppError(404, ...)`, que el middleware de errores convierte en una respuesta JSON. Si existe, el Controller responde con `response.json({ channel })`. La validación del ObjectId (400) ya venía preparada.

tv_hub_watch / [src/controllers/channel.controller.ts](#)

```
47 export const getChannel: RequestHandler = async (request, response) => {
48   const channelId = request.params.id;
49
50   if (!isValidObjectId(channelId)) {
51     throw new AppError(400, 'INVALID_CHANNEL_ID', 'Channel id is invalid');
52   }
53
54   // TODO 2:
55   // Completa el método de Mongoose que permite recuperar un solo canal activo.
56   // Objetivo: consultar el Channel Model utilizando el identificador recibido.
57   // Resultado esperado: channel debe contener el documento solicitado cuando exista.
58   const channel = await Channel.findOne({
59     _id: channelId,
60     isActive: true
61   });
62
63   // TODO 3:
64   // Completa el código HTTP cuando el canal solicitado no existe.
65   // Objetivo: comunicar correctamente que el recurso no fue encontrado.
66   // Resultado esperado: la API debe responder con el estado HTTP apropiado.
67   if (!channel) {
68     throw new AppError(
69       404,
70       'CHANNEL_NOT_FOUND',
71       'Channel was not found'
72     );
73   }
74
75   // TODO 4:
76   // Completa el método de Response que envía el Channel al frontend.
77   // Objetivo: regresar la información del canal en formato JSON.
78   // Resultado esperado: el cliente debe recibir un objeto con la propiedad channel.
79   response.json({ channel });
80 };
```

Código 2 · TODOs 2, 3 y 4: consulta con `findOne`, 404 cuando no existe y respuesta JSON.

```
localhost:3000/api/channels/6aba90e6893465c99e243cfb

Impresión con formato estilístico ☒

{
  "channel": {
    "_id": "6aba90e6893465c99e243cfb",
    "country": "Japan",
    "tvId": "Weathernews.jp@SD",
    "_v": 0,
    "categories": [
      "Weather"
    ],
    "createdAt": "2026-09-28T16:08:06.041Z",
    "isActive": true,
    "logoUrl": "https://i.imgur.com/A8uRSTS.png",
    "name": "Weathernews (720p)",
    "streamType": "hls",
    "streamUrl": "https://rch01e-alive-hls.akamaized.net/38fb45b25cdb05a1/out/v1/4e907bfabc684a1dae10df8431a84d21/index.m3u8",
    "updatedAt": "2026-09-28T16:08:06.041Z"
  }
}
```

Evidencia 2.1 · Respuesta JSON en el navegador: el objeto channel recuperado de MongoDB, con name, country, categories y streamUrl.

```
localhost:3000/api/channels/00000000000000000000000000000000

Impresión con formato estilístico ☐

{"error":{"code":"CHANNEL_NOT_FOUND","message":"Channel was not found"}}
```

Evidencia 2.2 · id válido que no existe → 404.

```
localhost:3000/api/channels/abc123

Impresión con formato estilístico ☒

{
  "error": {
    "code": "INVALID_CHANNEL_ID",
    "message": "Channel id is invalid"
  }
}
```

Evidencia 2.3 · id con formato inválido → 400.

```
Terminal — tv_hub_watch · MongoDB 8.2 en Docker

$ docker exec -it tvhub-mongo mongosh tvhub
tvhub> db.version()
8.2.12
tvhub> db.channels.countDocuments({ isActive: true })
159
tvhub> db.channels.findOne({ name: /NHK/ }, { name: 1, country: 1, categories: 1, streamUrl: 1, isActive: 1 })
{
  _id: ObjectId('6aba90e6893465c99e243cf8'),
  country: 'Japan',
  categories: [
    'General'
  ],
  isActive: true,
  name: 'NHK General TV (720p)',
  streamUrl: 'https://nhk4.mov3.co/hls/nhk.m3u8'
}
```

Evidencia 2.4 · Los mismos documentos consultados directamente en MongoDB con mongosh: es la fuente de datos del Model.

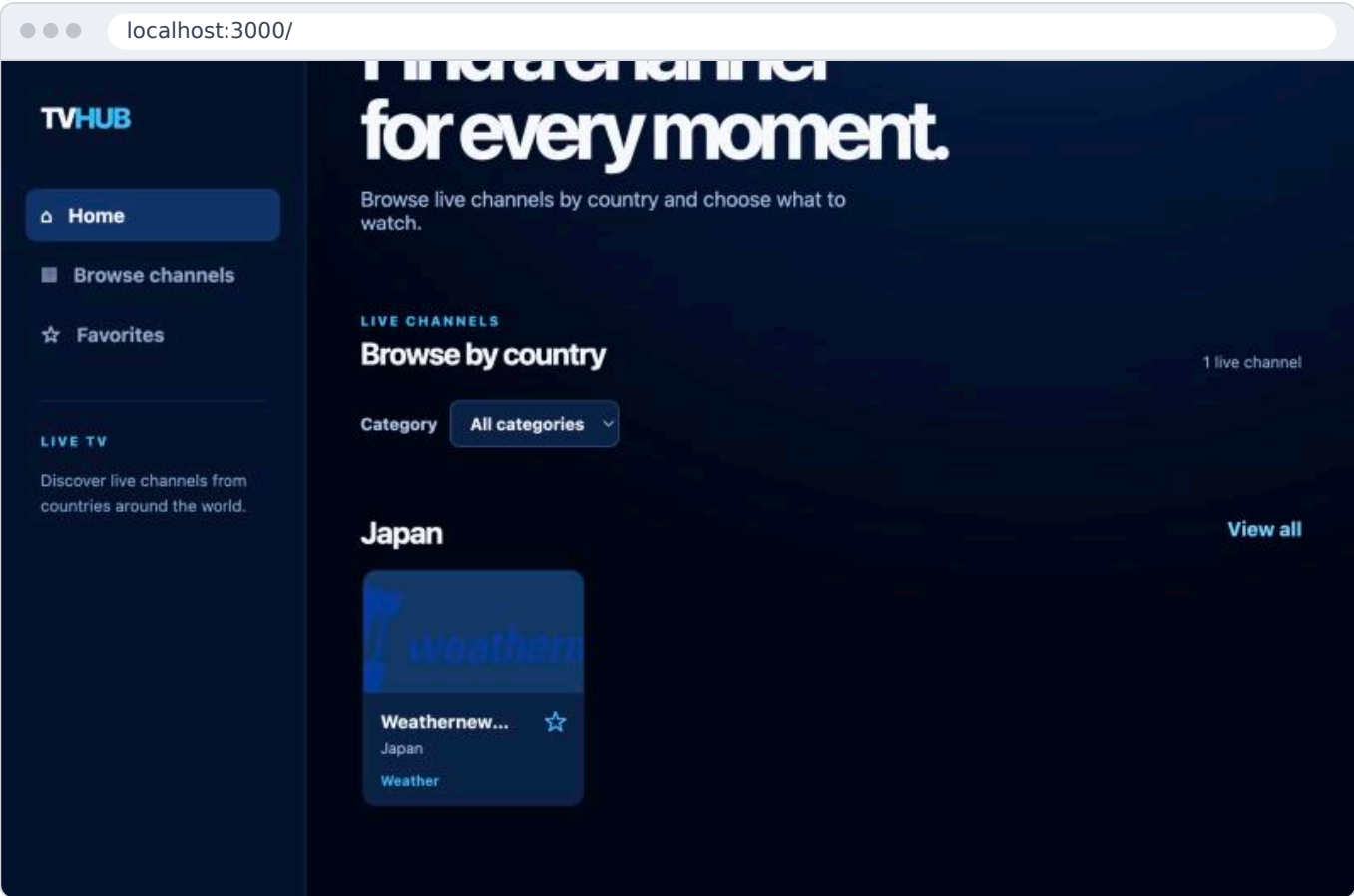
Ejercicio 3 · View y fetch

Qué se hizo. `watch.js` ya leía `channelId` de la query string (Home navega a `/watch.html?channelId=...`). Se construyó el endpoint `/api/channels/${channelId}` para el `fetch`, con `encodeURIComponent` para no romper la URL. Con el JSON recibido se actualiza el DOM usando `channel.name`, `channel.country` y `channel.categories`. La View nunca toca MongoDB: se comunica con el backend solo por HTTP.

tv_hub_watch / [src/public/js/watch.js](#)

```
50 async function loadChannel() {
51   if (!channelId) { showPlayerState('error', 'Choose a channel from Home.')} return; }
52
53   // TODO 5:
54   // Completa la URL utilizada para obtener el Channel seleccionado.
55   // Objetivo: conectar la View Watch con GET /api/channels/:id.
56   // Resultado esperado: DevTools debe mostrar una petición GET con respuesta 200.
57   const response = await fetch(`/api/channels/${encodeURIComponent(channelId)}`);
58   if (!response.ok) { showPlayerState('error', 'This channel is not available.')} return; }
59   ({ channel } = await response.json());
60   const logo = document.querySelector('#channel-logo'); logo.src = channel.logoUrl || '/images/channel-placeholder.svg';
   logo.alt = `${channel.name} logo`; logo.addEventListener('error', () => { logo.src = '/images/channel-placeholder.svg';
   });
61
62   // TODO 6:
63   // Completa las propiedades del Channel utilizadas por la View.
64   // Objetivo: mostrar la información recibida desde el backend.
65   // Resultado esperado: Watch debe mostrar nombre, país y categorías del canal seleccionado.
66   document.querySelector('#channel-name').textContent = channel.name;
67   document.querySelector('#channel-country').textContent = channel.country;
68   document.querySelector('#channel-categories').textContent = channel.categories.join(', ') || 'Live TV · No categories
   assigned yet';
69   await loadFavoriteState(); await playChannel();
70 }
```

Código 3 · TODOS 5 y 6: fetch a la API y actualización del DOM con las propiedades del Channel.



Evidencia 3.1 · El usuario busca y selecciona el canal Weathernews en Home.

Network · peticiones a localhost:3000 al abrir localhost:3000/watch.html?channelId=6aba90e6893465c99e243cfb				
Name	Method	Status	Type	Path
watch.html?channelId=6aba90...	GET	200	document	/watch.html?channelId=6aba90e6893465c99e243cfb
watch.js	GET	304	script	/js/watch.js
me	GET	200	fetch	/api/users/me
6aba90e6893465c99e243cfb	GET	200	fetch	/api/channels/6aba90e6893465c99e243cfb
favorites	GET	200	fetch	/api/favorites
Headers Request URL: http://localhost:3000/api/channels/6aba90e6893465c99e243cfb Request Method: GET Status Code: 200 OK Content-Type: application/json		Response <pre>{ "channel": { "_id": "6aba90e6893465c99e243cfb", "country": "Japan", "tvId": "Weathernews.jp@SD", "__v": 0, "categories": ["Weather"], "createdAt": "2026-09-28T16:08:06.041Z", "isActive": true, "logoUrl": "https://i.imgur.com/A8uRSTS.png", "name": "Weathernews (720p)", "streamType": "hls", "streamUrl": "https://rch01e-alive-hls.akamaized.net/38fb45b25cdb05a1/out/v1/4e907bfabc684a1dae10df8431a84d21/index.m3u8", "updatedAt": "2026-09-28T16:08:06.041Z" } }</pre>		

Evidencia 3.2 · Peticiones de red al abrir Watch: GET /api/channels/:id responde 200 con el Channel en JSON.

Ejercicio 4 · Player

Qué se hizo. Se creó el reproductor con `new shaka.Player(video)`, asociado al elemento `<video id="video-player">` que ya existía en `watch.html`, y se cargó el stream con `player.load(channel.streamUrl)`. El único dato del Channel que Shaka necesita es `streamUrl`; `name` y `country` solo se usan en la interfaz. Si ya había un player, se destruye antes de crear otro, así *Try again* puede volver a llamar a `playChannel()` sin duplicarlo.

tv_hub_watch / [src/public/js/watch.js](#)

```
29 async function playChannel() {
30   showPlayerState('loading', 'Preparing the live stream...');
31   if (!window.shaka) { showPlayerState('error', 'The player could not load. Try again.');
```

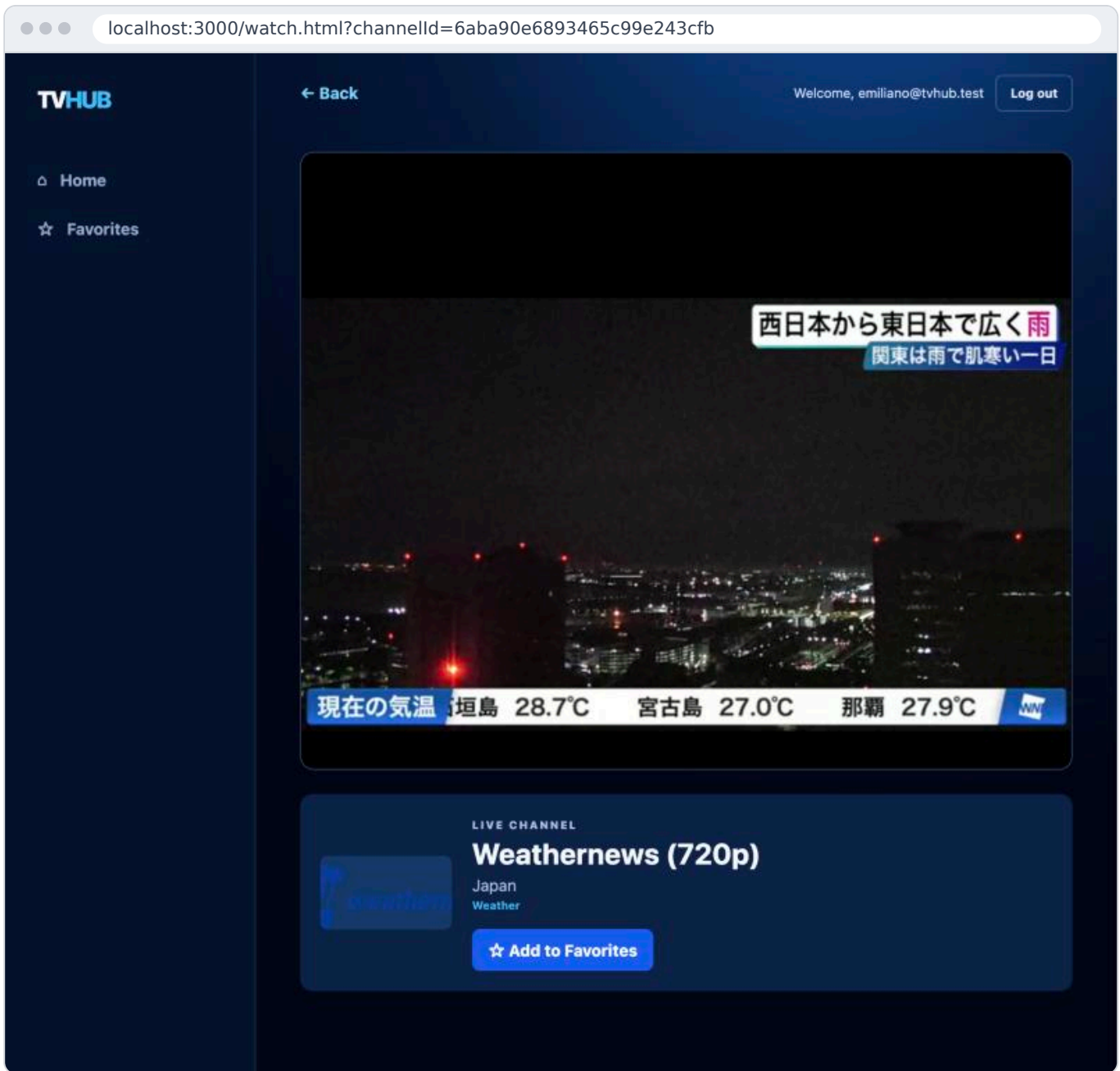
`return;`

```
;}
32   if (player) await player.destroy();
33   // TODO 7:
34   // Completa la integración básica con Shaka Player.
35   // Objetivo: asociar el Player con el elemento de video y cargar el stream del Channel.
36   // Resultado esperado: un stream disponible debe reproducirse en la página Watch.
37   player = new shaka.Player(video);
38   player.addEventListener('error', () => showPlayerState('error', 'This live stream cannot be played right now.));
39   try {
40     await player.load(channel.streamUrl);
41
42     // TODO 8:
43     // Completa los estados que debe mostrar el reproductor.
44     // Objetivo: comunicar si el stream se reprodujo correctamente o si ocurrió un error.
45     // Resultado esperado: la interfaz debe cambiar de Loading a Playing o Error.
46     showPlayerState('playing', '');
47     try { await video.play(); } catch { playerStatus.textContent = 'Press play to start audio.'; }
48   } catch { showPlayerState('error', 'This live stream cannot be played right now.');
```

`}`

```
}
```

Código 4 · TODO 7: Player asociado al video y carga de `channel.streamUrl`.



Evidencia 4.1 · Weathernews (Japan · Weather) reproduciéndose en Watch con Shaka Player. Debajo del video se ven el nombre, el país y la categoría que regresó la API.

Ejercicio 5 · Estados

Qué se hizo. `showPlayerState()` ya existía: cambia el mensaje y oculta o muestra el video y el botón *Try again*. Se completaron los estados según el resultado de `player.load()` :

Estado	Cuándo se usa	Qué ve el usuario
loading	Al entrar a <code>playChannel()</code> , antes de cargar el stream	“Preparing the live stream...”
playing	Cuando <code>await player.load()</code> termina sin excepción	El video visible, sin mensaje
error	En el <code>catch</code> de <code>load()</code> o en el evento <code>error</code> de Shaka	Mensaje de error y botón <i>Try again</i>

tv_hub_watch / [src/public/js/watch.js](#)

```
29 async function playChannel() {
30   showPlayerState('loading', 'Preparing the live stream...');
31   if (!window.shaka) { showPlayerState('error', 'The player could not load. Try again.');
```

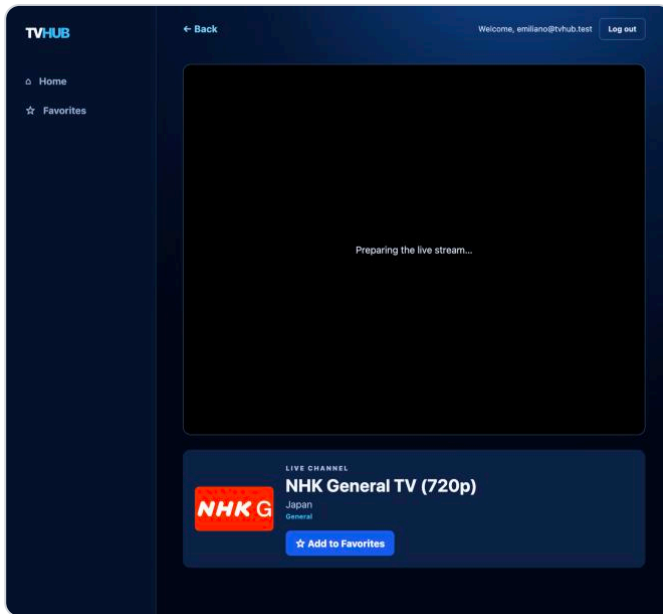
`return;`

```
32   if (player) await player.destroy();
33   // TODO 7:
34   // Completa la integración básica con Shaka Player.
35   // Objetivo: asociar el Player con el elemento de video y cargar el stream del Channel.
36   // Resultado esperado: un stream disponible debe reproducirse en la página Watch.
37   player = new shaka.Player(video);
38   player.addEventListener('error', () => showPlayerState('error', 'This live stream cannot be played right now.));
39   try {
40     await player.load(channel.streamUrl);
41
42     // TODO 8:
43     // Completa los estados que debe mostrar el reproductor.
44     // Objetivo: comunicar si el stream se reprodujo correctamente o si ocurrió un error.
45     // Resultado esperado: la interfaz debe cambiar de Loading a Playing o Error.
46     showPlayerState('playing', '');
47     try { await video.play(); } catch { playerStatus.textContent = 'Press play to start audio.'; }
48   } catch { showPlayerState('error', 'This live stream cannot be played right now.');
```

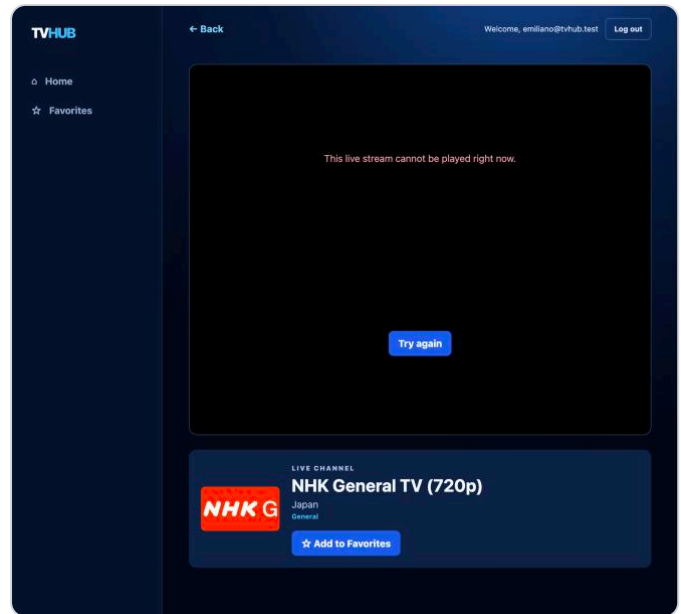
`}`

```
49 }
```

Código 5 · TODO 8: loading antes de cargar, playing al terminar y error en el catch.



Evidencia 5.1 · Estado **loading**: la API ya devolvió el canal y Shaka está cargando el stream.



Evidencia 5.2 · Estado **error**: el stream de NHK General TV no responde (Shaka error 1002, HTTP). La API sí devolvió el canal, y aparece Try again.

El estado **playing** se ve en la Evidencia 4.1 (Weathernews). Así se distinguen los tres estados: la API respondió 200 en ambos canales, y la diferencia está solo en la disponibilidad del stream.

Opcionales

Try again: el botón está conectado con `retryButton.addEventListener('click', playChannel)`. Al pulsarlo se vuelve a pasar por loading → playing/error. **Favoritos:** Watch reutiliza la API de Favorites: `POST /api/favorites/:channelId` crea el favorito y `DELETE` lo elimina. El método HTTP expresa la intención y la URL identifica el recurso. **Sin categorías:** si `categories` viene vacío, la UI muestra “Live TV · No categories assigned yet” en lugar de un texto vacío.

Conclusiones

En esta práctica vi MVC funcionando en un proyecto real, siguiendo una sola acción del usuario de principio a fin: elegir un canal y verlo.

View (`watch.html` y `watch.js`): es lo que ve el usuario. No sabe que existe MongoDB; solo pide datos por HTTP con `fetch` y pinta lo que recibe. También maneja la experiencia: los estados loading, playing y error, y el reproductor.

Route (`channel.routes.ts`): relaciona `GET /api/channels/:id` con `getChannel` y no contiene lógica de negocio.

Controller (`getChannel`): recibe la petición, valida el id, le pide al Model el canal y decide la respuesta: 200 con JSON, 404 si no existe o 400 si el id es inválido.

Model (`Channel` con Mongoose): define la forma del documento (name, country, categories, streamUrl, isActive...) y ofrece métodos como `findOne()` para consultarlo sin escribir la consulta a mano.

MongoDB: guarda los canales importados desde las playlists M3U. Es la fuente de verdad del flujo; el mismo documento que se ve con mongosh es el que llega en JSON a la View.

Lo más útil fue ver que cada capa se puede probar por separado. Primero confirmé que la API respondía JSON (Route, Controller y Model), después que Watch mostraba los datos (View) y al final el player. Con NHK el video no cargó, pero como la API ya había respondido 200 supe que el problema estaba en el stream y no en el backend. Separar responsabilidades hace mucho más fácil ubicar dónde está un error.